Autonomous Invariant-Preserving Deadlock-Free Semantic Resource Arbitration and Epistemic Priority Inheritance for Multi-Agent Transactional Databases

Aniket Agrawal, *Google Cloud India Private Limited*

Abstract—The deployment of autonomous multi-agent swarms operating over shared enterprise transactional databases introduces a fundamental concurrency crisis. Unlike classical database transactions characterized by sub-millisecond compiled SQL queries, autonomous agents exhibit high-latency semantic reasoning phases (250 ms to 2,500 ms per step; 500 ms to 5,000 ms across multi-step workflows) while evaluating state-dependent domain policies, invoking external compliance tools, and executing multi-step LLM prompts within active critical sections. Under standard Strict Two-Phase Locking (S2PL), overlapping multi-resource lock acquisitions generate closed cycles in the system's Wait-For-Graph ($\mathcal{G}_{\text{wait}}$), while timestamp-based prevention (Wait-Die, Wound-Wait) and Optimistic Concurrency Control (OCC) trigger frequent mid-execution or commit-time aborts. Crucially, in multi-agent swarms, transaction aborts induce severe *livelock token thrashing*, as aborted agents re-execute stochastic planning paths and discard hundreds to thousands of already-billed LLM inference tokens.

To resolve this systems bottleneck within a centralized shared-memory transactional lock coordinator or connection-pool gateway fronting an OLTP engine (such as a single Spanner Paxos group leader's local lock table), we present the **Epistemic Priority Ceiling Protocol (EPCP)**, an autonomous, invariant-preserving, provably deadlock-free concurrency control architecture. EPCP establishes a monotonically latched dynamic priority metric $\tilde{\Pi}_i(t)$ incorporating agent task tree depth, accumulated LLM token investment with aging, and inverse epistemic entropy, paired with a static upper-envelope preemption level $\Pi_i^{\max}$ over exact 64-bit integer lexicographical priority tuples. By maintaining fine-grained 64-partition intent-footprint priority ceilings and enforcing a guarded preemptive admission invariant ($\Pi_i^{\text{eff}}(t) > C^*_{-A_i}(t; \mathcal{F}(T_i))$), EPCP serializes admission across intersecting dynamic footprints before critical sections are entered while allowing disjoint footprints to execute in parallel without aborting in-flight transactions. We provide a rigorous inductive proof establishing that $\mathcal{G}_{\text{wait}}(t)$ remains a directed acyclic graph (DAG) under all thread interleavings. We implemented EPCP natively on macOS Apple Silicon and benchmarked sustained multi-threaded database transactions over an enterprise banking and supply chain schema (10,000 Accounts, 5,000 Inventory Items) under heavy Zipfian access skew ($\theta = 0.85$) across sub-millisecond cryptographic HMAC-SHA256 stress tests, 500 ms–1,500 ms multi-step LLM reasoning benchmarks, and live Google Cloud Vertex AI `gemini-2.5-flash` multi-agent runs against S2PL, 2PL-Wait Promote, Wound-Wait, Wait-Die, OCC, Coarse Shard-Scoped C2PL, and Canonical Ordered Locking. Empirical results demonstrate that EPCP achieves a **0.00% deadlock and concurrency abort rate** across all regimes, completely eliminating victim transaction rollbacks while preserving strict conflict serializability and physical ACID integrity.

Index Terms—Transactional Databases, Multi-Agent Concurrency Control, Deadlock Elimination, Priority Ceiling Protocol, Epistemic Uncertainty, Cloud Spanner, Transaction Processing.

1 Introduction

Enterprise cloud platforms are undergoing a paradigm shift from human-in-the-loop task execution to autonomous multi-agent systems. Built on orchestration frameworks such as Google Agent Development Kit (ADK) and Vertex AI Agent Builder, autonomous agent swarms are delegated end-to-end responsibilities over core enterprise workflows, including supply chain reconciliation, automated loan underwriting, financial clearing, and algorithmic procurement. In these architectures, multiple autonomous agents execute concurrently through a centralized shared-memory transactional lock coordinator or connection-pool gateway fronting shared OLTP database engines such as a single Google Cloud Spanner Paxos group leader [6], AlloyDB/PostgreSQL, or CockroachDB (leaving decentralized cross-leader two-phase commit ceiling synchronization across geo-distributed consensus groups to future work).

1.1 The Semantic Reasoning Latency Expansion and Why Locks Span Inference

Classical transaction processing was developed under the assumption that transactions are short-lived, pre-compiled SQL queries with execution latencies on the order of 100 microseconds to 5 milliseconds [1], [2]. Under this microsecond regime, locking protocols such as Strict Two-Phase Locking (S2PL) [3] provide conflict serializability with negligible lock contention windows.

In autonomous multi-agent architectures, however, transactions are inherently interleaved with *state-dependent*

• *A. Agrawal is an AI Cloud Engineer at Google Cloud India Private Limited, Bangalore, India (e-mail: aniketagrawal@google.com).*

semantic reasoning loops. A natural question is whether holding locks across LLM inference can be avoided altogether via (i) snapshot reads followed by short compare-and-swap (CAS) / Optimistic Concurrency Control (OCC) validation at commit, (ii) simple commutative escrow columns (e.g., `reserved_amount`), or (iii) Saga-based non-isolated sub-transactions [10]. In regulated enterprise workflows, each of these alternatives breaks down under high contention:

1) **Failure of Snapshot-Read + CAS and Scalar Escrow on Non-Commutative State Mutations:** While simple scalar decrements can use commutative escrow counters, enterprise agent workflows perform *non-commutative, multi-attribute structural mutations*—such as corporate loan covenant restructuring, cross-collateralized margin re-indexing, and serialized lot compliance—where Step 2's multi-turn LLM synthesis, AML polynomial risk scoring, and generated SQL/JSON compliance patch depend nonlinearly on the exact vector of row attributes read in Step 1. When an agent acquires a pessimistic lock on R_a, it first runs an immediate submicrosecond scalar sufficiency pre-filter (`Biz-Rule Reject`) to fast-exit obviously depleted accounts or out-of-stock SKUs with zero wasted LLM tokens; for funded rows passing this pre-filter, the subsequent Infer(ϕ) phase evaluates the full multi-attribute state of R_a. Under snapshot-read + CAS (OCC), if any contending agent modifies R_a during the 500 ms–5,000 ms multi-step reasoning window, the generated compliance patch is invalidated and CAS fails at commit time *after 100% of the multi-step LLM token budget has already been expended*. Holding a pessimistic lock across Infer(ϕ) protects funded rows from mid-inference invalidation.
2) **Prohibition of Uncommitted State Exposure:** Saga-based agent workflows [10] release locks early by committing intermediate sub-transactions and relying on reverse compensating actions upon failure. However, in strict double-entry banking and regulatory clearing, exposing transient intermediate states to concurrent readers violates strict serializability and triggers cascading audit exceptions.

Consequently, preserving strict ACID isolation without post-hoc validation rollbacks on non-commutative multi-attribute workflows requires holding pessimistic locks across intermediate reasoning phases, expanding lock retention durations from milliseconds to several seconds (500 ms to 5,000 ms)—a $10^3\times$ to $10^4\times$ duration expansion.

1.2 The Livelock and Token Thrashing Crisis

When agents acquire locks across multiple resources in dynamic orderings (e.g., Agent A_1 acquiring Account $\rightarrow$ Inventory while Agent A_2 acquires Inventory $\rightarrow$ Account), Wait-For-Graph ($\mathcal{G}_{\text{wait}}$) cycles form rapidly. Under S2PL, deadlock detection modules identify cycles and forcibly abort a "victim" transaction [1]. Similarly, classical timestamp-ordered deadlock prevention schemes [5] avoid cycles by aborting transactions whenever timestamp inversions occur: under non-preemptive *Wait-Die*, a younger requester immediately "dies" (aborts) upon encountering an older lock holder while an older requester waits; under preemptive *Wound-Wait* (used in Google Cloud Spanner [6]), an older requester immediately "wounds" (preemptively aborts) a younger lock holder while a younger requester waits.

In traditional relational databases, aborting an in-memory SQL transaction incurs minimal penalty. In multi-agent swarms, however, aborting an in-flight agent transaction—whether via S2PL deadlock victim selection, Wait-Die self-abort, Wound-Wait preemption [5], or OCC validation failure—results in **catastrophic token thrashing**:

- **Token & Financial Waste:** An agent rolled back mid-workflow loses its in-flight transactional state, discarding hundreds to thousands of already-billed LLM inference tokens (e.g., on Gemini 2.5 Pro / Flash endpoints).
- **Stochastic Lock Retrying:** Upon restart, the agent re-evaluates prompts against mutated state, frequently re-colliding with contending agents.
- **System Saturation:** Under heavy partition skew, abort-based concurrency controllers suffer persistent rollback cascades.

Real-time concurrency control offers foundational mechanisms for deadlock-free execution without victim aborts. Priority Inheritance and Priority Ceiling Protocols (PCP) were pioneered by Sha, Rajkumar, and Lehoczky [4] for uniprocessor tasks with static priorities, and extended to dynamic-priority real-time tasks (such as Earliest Deadline First) by Chen and Lin's Dynamic Priority Ceiling Protocol [7] and Baker's Stack Resource Policy (SRP) [8], which decouple static resource preemption levels from dynamic task urgency. In real-time databases, Abbott and Garcia-Molina [9] evaluated priority-cognizant 2PL variants—*2PL-High Priority* (which aborts lower-priority lock holders, inducing token waste) and *2PL-Wait Promote* (which applies priority inheritance upon direct row lock conflicts). Crucially, because *2PL-Wait Promote* lacks a preemptive partition priority ceiling admission gate, transactions still acquire initial locks in opposing orders and suffer multi-resource $\mathcal{G}_{\text{wait}}$ deadlocks. In parallel, classical database deadlock prevention via Conservative 2PL (C2PL) [2], canonical ordered locking, or deterministic scheduling such as Calvin [11] requires transactions to either pre-declare and lock every exact row key upfront or sort all target row IDs prior to acquiring any lock. However, in multi-step agentic workflows, the fine-grained target row key R_b in Step 2 (e.g., which specific warehouse lot or counterparty ledger row to debit) is discovered dynamically from the output of Step 1's LLM inference Infer(ϕ), making exact row-level C2PL or upfront canonical row sorting impossible without either locking coarse domain shards exclusively or running an unisolated speculative discovery pre-pass. Recent agentic concurrency frameworks—including SagaLLM [10] for compensating workflows, ATCC [12] (which adaptively switches LLM agent transactions between optimistic and pessimistic concurrency control based on conflict rates), and CoAgent [13] (which co-designs multi-agent execution scheduling and database concurrency control)—improve agent throughput under contention, but rely on optimistic/pessimistic aborts

or schedule reordering rather than a provably deadlock-free, non-aborting priority ceiling protocol coupled to live LLM token investment and epistemic action entropy.

1.3 Contributions

In this work, we present the **Epistemic Priority Ceiling Protocol (EPCP)**, an autonomous, invariant-preserving (in the concurrency-control sense of maintaining cycle-free wait invariants dynamically without background deadlock detectors or victim aborts) concurrency control architecture engineered for high-latency semantic agent transactions. Our primary contributions are:

1) **Dual-Level Epistemic Priority Architecture ($\Pi_i^{\max}$ and $\tilde{\Pi}_i(t)$):** A formal priority formulation coupling task delegation depth (d_i), clipped cumulative token and aging expenditure ($\Omega_i(t)$), and inverse normalized token-level Shannon action entropy ($1 - \mathcal{H}_i(t)$, where $\mathcal{H}_i(t) \in [0,1]$ measures action distribution uncertainty), represented as exact 64-bit integer lexicographical tuples to guarantee strict total ordering without floating-point precision loss.
2) **Intersecting-Footprint Partition Priority Ceilings ($C^*_{-A_i}(t; \mathcal{F}(T_i))$):** A 64-partition lock admission invariant where an agent declares an ADK routing-header hash footprint $\mathcal{F}(T_i) \subseteq \{1, \ldots, 64\}$ at registration, serializing initial entry across intersecting dynamic footprints ($\mathcal{F}(T_i) \cap \mathcal{F}(T_k) \neq \emptyset$) to prevent Step-2 dynamic-discovery cycles while executing disjoint footprints ($\mathcal{F}(T_i) \cap \mathcal{F}(T_k) = \emptyset$) in full parallel concurrency.
3) **Provable Deadlock Immunity (Theorem 1):** A rigorous inductive proof over unified contention and ceiling wait edges demonstrating that $\mathcal{G}_{\text{wait}}(t)$ remains a directed acyclic graph (DAG) at all times, guaranteeing a 0.00% deadlock abort rate without victim selection.
4) **Bounded Priority Inversion & Starvation Freedom (Theorems 2 & 3):** Proofs establishing per-partition single-critical-section inversion bounds via transitive priority inheritance and unconditional finite-time completion via atomic senior saturation promotion.
5) **Comprehensive Empirical & Live Vertex AI Validation:** Multi-threaded benchmarks over an enterprise banking and supply chain schema (10,000 Accounts, 5,000 Inventory Items) under Zipfian skew ($\theta = 0.85$), spanning sub-millisecond HMAC-SHA256 stress tests (with Zero-Depletion Controls across $K \in \{20, 30, 40\}$), 500 ms–1,500 ms 2-step LLM reasoning benchmarks (250–750 ms/step across 9 protocols including `2PL-Wait Promote`), and live Google Cloud Vertex AI `gemini-2.5-flash` multi-agent evaluations on project `aniket-personal`.

2 System Model and Problem Formulation

2.1 Execution Environment and Intent-Scoped Footprints

We consider a transactional database system $\mathcal{D}$ arbitrated by a centralized shared-memory lock coordinator (or a single Spanner Paxos group leader's local lock table [6]) maintaining a set of shared row-level resources $\mathcal{R} = \{R_1, R_2, \ldots, R_M\}$, mapped via a deterministic FNV-1a 64-bit hash function $\text{part} : \mathcal{R} \to \{1, \ldots, B_{\text{part}}\}$ ($B_{\text{part}} = 64$) into fine-grained logical lock partitions. Each transaction's partition footprint is represented compactly as a single 64-bit unsigned integer bitmask (`uint64_t`), enabling $O(1)$ hardware bitwise-`AND` footprint intersection checks. A swarm of K autonomous agent worker threads $\mathcal{A} = \{A_1, A_2, \ldots, A_K\}$ concurrently execute transactional tasks.

Unlike row-level C2PL (which requires knowing every exact row primary key R_a, R_b before locking), EPCP requires only an **Intent-Scoped Partition Footprint** $\mathcal{F}(T_i) \subseteq \{1, \ldots, B_{\text{part}}\}$ (typically $|\mathcal{F}(T_i)| \in \{2, 3\}$ out of $B_{\text{part}} = 64$), extracted statically from the mandatory routing metadata of the agent's ADK/MCP tool invocation schema (e.g., the 64-bucket hash IDs of the target customer account segment and warehouse SKU category). While $\mathcal{F}(T_i)$ is known at registration, the exact fine-grained row keys $R_a, R_b \in \mathcal{R}$ within $\mathcal{F}(T_i)$ are discovered dynamically during execution:

$$T_i = \langle \text{Register}(\mathcal{F}(T_i)),\ \text{Lock}(R_a),\ \text{BizCheck}(R_a),\ \text{Infer}(\phi),\ \text{StageWrite}(R_a),\ \text{Lock}(R_b),\ \ldots,\ \text{Commit} \rangle \tag{1}$$

where $\text{Infer}(\phi)$ denotes parameterized LLM policy inference (using ϕ to distinguish model parameters from the Zipfian skew parameter θ), and $\text{StageWrite}(R_a)$ stages mutations in a transaction-local write buffer that is only flushed atomically to physical database storage at Commit after all locks are held. If an agent ever attempts to request an out-of-footprint resource R_{req} with $\text{part}(R_{\text{req}}) \notin \mathcal{F}(T_i)$, the coordinator rejects the lock call, discards the uncommitted local write buffer (so zero physical database rollback is required), memoizes completed $\text{Infer}(\phi)$ outputs in the agent's local context cache keyed by a cryptographic SHA-256 digest of the read attributes of R_a (`state_version_digest`), releases held locks, expands $\mathcal{F}(T_i) \leftarrow \mathcal{F}(T_i) \cup \{\text{part}(R_{\text{req}})\}$, and re-registers *while preserving its original arrival timestamp $t_{start,i}$ and monotonic rank $rank(T_i)$*. Upon re-acquiring R_a, if R_a's attributes are unchanged (matching `state_version_digest`), the memoized inference output is replayed in microseconds with **zero re-billed LLM tokens**; if R_a was modified by another transaction during the brief unlock window, the SHA-256 digest misses and $\text{Infer}(\phi)$ is freshly re-evaluated against the new state (in our evaluated ADK banking and supply-chain tool schemas, routing headers bound 100% of candidate partitions upfront, so footprint expansion is never triggered).

2.2 Unified Wait-For-Graph and Deadlock Formalization

Let $\text{Held}(A_j, t) \subseteq \mathcal{R}$ denote the set of resources currently locked by agent A_j at time t.

Definition 1 (Unified Wait-For-Graph)**.** *The Unified Wait-For-Graph at time t, denoted $\mathcal{G}_{wait}(t) = (\mathcal{V}, \mathcal{E}(t))$, is a directed graph where vertices $\mathcal{V} = \mathcal{A}$ represent active agent worker threads, and a directed edge $(A_i \to A_j) \in \mathcal{E}(t)$ $(i \neq j)$ exists if and only if agent A_i is currently suspended awaiting lock acquisition on resource R_{req} ($part(R_{req}) \in \mathcal{F}(T_i)$) because:*

1) ***Direct Contention Wait:*** $R_{req} \in Held(A_j, t)$*; or*
2) ***Ceiling Preemption Wait:*** R_{req} *is free, Eq.* (11) *fails for* A_i*, and* A_j $(j \neq i)$ *is an active lock holder* $(Held(A_j, t) \neq \emptyset)$ *with an intersecting partition footprint* $(\mathcal{F}(T_i) \cap \mathcal{F}(T_j) \neq \emptyset)$ *whose shared partition ceiling blocks* A_i*'s admission* $(\max_{P_m \in \mathcal{F}(T_i) \cap \mathcal{F}(T_j)} \Pi(P_m, t) \geq \Pi_i^{eff}(t))$*.*

Definition 2 (Deadlock State)**.** *A system is in a deadlock state at time* t *if and only if there exists a directed cycle in* $\mathcal{G}_{wait}(t)$*:*

$$A_{p_1} \to A_{p_2} \to \cdots \to A_{p_L} \to A_{p_1} \tag{2}$$

Under S2PL, resolving this cycle requires selecting a victim agent A_{p_v} *and aborting its transaction, rolling back uncommitted writes and discarding in-flight LLM reasoning tokens.*

3 The Epistemic Priority Ceiling Protocol

3.1 Monotonically Latched Epistemic Priority Formulation

Unlike classical real-time operating systems where thread priorities are static integers [4], [8], autonomous agent priorities must reflect both the economic sunk cost and epistemic certainty of in-flight reasoning tasks while remaining strictly ordered and non-decreasing during an active transaction. Following Baker's SRP principle [8], EPCP maintains two complementary priority quantities per transaction T_i: (1) a static **Transaction Upper-Envelope Preemption Level** $\Pi_i^{\max}$ (determined at registration by task depth d_i and arrival rank $\text{rank}(T_i)$) that defines invariant partition ceilings so ceilings never shift mid-lock and cause self-deadlock, and (2) a live **Monotonically Latched Dynamic Priority** $\tilde{\Pi}_i(t)$ (driven by real-time token expenditure $\text{Tokens}_i(t)$, wait-time aging, and action certainty $1 - \mathcal{H}_i(t)$) that orders condition-variable wake-up queues and dynamic urgency scheduling among waiting agents. Thus, when a lock holder commits and signals blocked waiters, the waiting agent with the highest live epistemic investment $\tilde{\Pi}_k^{\text{eff}}(t)$ is dispatched first to acquire the released resource.

Definition 3 (Dynamic and Latched Epistemic Priority)**.** *At time* t*, the instantaneous epistemic score* $\Pi_i(t) \in \mathbb{R}^+$ *of agent* A_i *executing transaction* T_i *(started at* $t_{start,i}$*) is:*

$$\Pi_i(t) \triangleq \Pi_{base} + w_d \cdot d_i + w_\Omega \cdot \ln(1 + \Omega_i(t)) + w_c \cdot (1 - \mathcal{H}_i(t)) \tag{3}$$

where $\Pi_{base} = 1.0$*,* $d_i \in \{1, \ldots, d_{\max}\}$ *(*$d_{\max} = 4$*) is the agent call-graph delegation depth, and* $\Omega_i(t)$ *combines cumulative LLM token investment with linear wait-time aging (*$\gamma_{age} = 0.05\ s^{-1}$*, explicitly clipped at* $\Omega_{\max} = 10^4$*):*

$$\Omega_i(t) \triangleq \min\left(\Omega_{\max},\ \frac{Tokens_i(t)}{100} + \gamma_{age}(t - t_{start,i})\right) \tag{4}$$

with $\Omega_i(t) \in [0, \Omega_{\max}]$*. The normalized token-level Shannon entropy* $\mathcal{H}_i(t) \in [0, 1]$ *over the agent's candidate tool-action distribution* $p_{i,t}(v)$ $(v \in \mathcal{V}_{act}$*, extracted from the softmax log-probabilities of the LLM's structured tool-call head at time* t*, initialized to neutral uncertainty* $\mathcal{H}_i(t_{start,i}) = 0.5$ *prior to the first inference step) is defined as:*

$$\mathcal{H}_i(t) \triangleq \begin{cases} 0, & \text{if } |\mathcal{V}_{act}| \leq 1, \\ -\frac{1}{\ln |\mathcal{V}_{act}|} \sum_{v \in \mathcal{V}_{act}} p_{i,t}(v) \ln p_{i,t}(v), & \text{if } |\mathcal{V}_{act}| > 1, \end{cases} \tag{5}$$

with weights fixed at $w_d = 2.0, w_\Omega = 1.5, w_c = 3.0$ *(chosen so one delegation level* $w_d = 2.0$ *outweighs cold-start entropy variance while full epistemic certainty* $w_c = 3.0$ *and log-token investment* $w_\Omega \ln(1 + \Omega_i(t)) \in [0, 13.82]$ *dominate mid-flight urgency; sensitivity sweeps across* $\pm 50\%$ *weight perturbations preserve* 0.00% *aborts with* $< 2.1\%$ *goodput variation). Furthermore, to guarantee unconditional starvation freedom across arbitrary depths (Theorem 3), if a waiting transaction's age reaches the senior saturation horizon* $T_{sat} = \Omega_{\max}/\gamma_{age}$*, its latched score AND its upper-envelope preemption score are atomically promoted to the uniform saturation ceiling* $\Pi_{sat} \triangleq \Pi_{base} + w_d \cdot d_{\max} + w_\Omega \cdot \ln(1 + \Omega_{\max}) + w_c$*, updating* $\Pi(P_m, t)$ *for all* $P_m \in \mathcal{F}(T_i)$ *and immediately propagating the promoted priority to any blocking holders via Eq.* (12)*.*

Let $\Pi_i^{score}(t) \triangleq \max_{\tau \in [t_{start,i}, t]} \Pi_i(\tau)$ *denote the monotonically latched dynamic score and* $\Pi_{i,\max}^{score} \triangleq \Pi_{base} + w_d d_i + w_\Omega \ln(1 + \Omega_{\max}) + w_c$ *(upgraded to* Π_{sat} *if* $t - t_{start,i} \geq T_{sat}$*) denote the upper-envelope preemption score. To eliminate IEEE-754 floating-point mantissa precision loss or negativity as the monotonic arrival sequence ID* $\text{rank}(T_i) \in \mathbb{N}$ *grows over millions of transactions, the coordinator represents priorities internally as exact 64-bit integer lexicographical pairs* $\tilde{\mathbf{\Pi}}_i(t) = \left(\lfloor 10^6 \Pi_i^{score}(t) \rfloor, -rank(T_i)\right)$ *and* $\mathbf{\Pi}_i^{\max} = \left(\lfloor 10^6 \Pi_{i,\max}^{score} \rfloor, -rank(T_i)\right) \in \mathbb{Z}_{\geq 0} \times \mathbb{Z}_{<0}$ *(and rebases* $rank(T_i) \leftarrow 1$ *whenever* $\mathcal{A}_{active}(t) = \emptyset$*). In scalar notation, we write this strict lexicographical order via the tie-breaker* $\epsilon_{tie} = 10^{-7}$ *as the* ***Monotonically Latched Dynamic Priority*** $\tilde{\Pi}_i(t)$ *and the* ***Transaction Upper-Envelope Preemption Level*** $\Pi_i^{\max}$*:*

$$\tilde{\Pi}_i(t) \triangleq \Pi_i^{score}(t) - \epsilon_{tie} \cdot rank(T_i), \tag{6}$$

$$\Pi_i^{\max} \triangleq \Pi_{i,\max}^{score} - \epsilon_{tie} \cdot rank(T_i) \tag{7}$$

By construction, for all $t_1 \leq t_2$ *during* T_i*'s execution,* $0 < \tilde{\Pi}_i(t_1) \leq \tilde{\Pi}_i(t_2) \leq \Pi_i^{\max}$*, and* $\Pi_i^{\max} \neq \Pi_j^{\max}$ *for all* $i \neq j$*.*

3.2 Atomic Registration and Intersecting-Footprint Partition Ceilings

Let $\mathcal{A}_{\text{active}}(t)$ denote the set of registered active transactions at time t. In EPCP, Register$(\mathcal{F}(T_q))$ and T_q's initial lock check Lock(R_a) execute as a **single atomic critical section under the coordinator mutex**: when T_q arrives at time t_{arr} with declared footprint $\mathcal{F}(T_q)$, T_q is inserted into $\mathcal{A}_{\text{active}}(t)$ and into $\mathcal{U}(P_m, t) \triangleq \{A_r \in \mathcal{A}_{\text{active}}(t) \mid P_m \in \mathcal{F}(T_r)\}$ for all $P_m \in \mathcal{F}(T_q)$ (elevating each partition ceiling $\Pi(P_m, t)$ via the exact 64-bit integer tuple comparison $\mathbf{\Pi}_q^{\max}$), and immediately evaluates Eq. (11) for R_a. If any active lock holder A_k $(\text{Held}(A_k, t_{\text{arr}}) \neq \emptyset)$ has an intersecting footprint $\mathcal{F}(T_k) \cap \mathcal{F}(T_q) \neq \emptyset$, T_q atomically binds ceiling wait edges $(A_q \to A_k)$ for all $A_k \in \mathcal{B}_{\text{ceil}}(A_q, t_{\text{arr}})$ and propagates $\Pi_q^{\max}$ to A_k via transitive priority inheritance (Eq. (12)) before releasing the coordinator mutex to sleep on CV_{ceil}.

Definition 4 (Partition Ceiling and Intersecting-Footprint System Ceiling)**.** *For each logical partition* $P_m \in \{1, \ldots, B_{part}\}$*, the partition priority ceiling* $\Pi(P_m, t)$ *(and resource ceiling* $C(R_j, t) \triangleq \Pi(part(R_j), t)$*) is the maximum upper-envelope preemption bound among all active transactions registered for partition* P_m*:*

$$\Pi(P_m, t) \triangleq \max\left(\{0\} \cup \{\Pi_q^{\max} \mid A_q \in \mathcal{U}(P_m, t)\}\right) \tag{8}$$

*Because $\Pi(P_m, t)$ is computed over the upper-envelope $\Pi_q^{\max}$ of registered agents, $\Pi(P_m, t) \geq \tilde{\Pi}_q(\tau)$ holds for every $A_q \in \mathcal{U}(P_m, t)$ at all instants τ. For an agent A_i with footprint $\mathcal{F}(T_i)$, let $\mathcal{K}_{ext}(A_i, t) \triangleq \{A_k \in \mathcal{A}_{active}(t) \setminus \{A_i\} \mid Held(A_k, t) \neq \emptyset \wedge \mathcal{F}(T_k) \cap \mathcal{F}(T_i) \neq \emptyset\}$ denote the set of active external lock-holding agents whose declared partition footprint intersects $\mathcal{F}(T_i)$. The **External Intersecting-Footprint System Ceiling** $C^*_{-A_i}(t; \mathcal{F}(T_i))$ and the **Overall Intersecting-Footprint System Ceiling** $C^*(t; \mathcal{F}(T_i))$ are defined as:*

$$C^*_{-A_i}(t; \mathcal{F}(T_i)) \triangleq \max\Big(\{0\} \cup \{\Pi(P_m, t) \mid A_k \in \mathcal{K}_{ext}(A_i, t), P_m \in \mathcal{F}(T_i) \cap \mathcal{F}(T_k)\}\Big), \tag{9}$$

$$C^*(t; \mathcal{F}(T_i)) \triangleq \max\Big(\{0\} \cup \{\Pi(P_m, t) \mid A_k \in \mathcal{A}_{active}(t), Held(A_k, t) \neq \emptyset,\ P_m \in \mathcal{F}(T_i) \cap \mathcal{F}(T_k)\}\Big) \tag{10}$$

Notice two fundamental properties of Eq. (9)*:*

1) ***Parallel Concurrency Across Disjoint 64-Bucket Footprints ($\mathcal{K}_{ext}(A_i, t) = \emptyset$):*** *Whenever no external lock-holding transaction has a footprint intersecting $\mathcal{F}(T_i)$ ($\mathcal{K}_{ext}(A_i, t) = \emptyset$, e.g., when concurrent agents operate on non-overlapping hash buckets across $B_{part} = 64$), Eq.* (9) *takes the maximum over the empty set and yields $C^*_{-A_i}(t; \mathcal{F}(T_i)) = 0 < \Pi_i^{eff}(t)$. Thus, A_i never self-blocks on its own registration in $\mathcal{U}(P_m, t)$ and enters immediately in parallel with all disjoint-footprint transactions.*
2) ***Intersecting-Footprint Admission Exclusion ($\mathcal{K}_{ext}(A_i, t) \neq \emptyset$):*** *Because $A_i \in \mathcal{U}(P_m, t)$ for all $P_m \in \mathcal{F}(T_i)$, whenever an external transaction $A_k \in \mathcal{K}_{ext}(A_i, t)$ already holds any lock ($Held(A_k, t) \neq \emptyset$) and shares at least one partition $P_{shared} \in \mathcal{F}(T_i) \cap \mathcal{F}(T_k)$, Eq.* (9) *yields $C^*_{-A_i}(t; \mathcal{F}(T_i)) \geq \Pi(P_{shared}, t) \geq \max(\Pi_i^{\max}, \Pi_k^{\max}) \geq \Pi_i^{\max}$. Because Step-2 row keys inside $\mathcal{F}(T_i)$ are discovered dynamically during Step-1 LLM inference, this intentionally enforces **single-holder admission exclusion across intersecting footprints**: a newly arriving A_i ($Held(A_i, t) = \emptyset$, even if $\Pi_i^{\max} > \Pi_k^{\max}$) is suspended at the admission gate $CV_{ceil}(P_{shared})$ rather than entering P_{shared} concurrently with A_k (which would risk A_i and A_k dynamically discovering and requesting each other's held rows in Step 2), and instead immediately elevates A_k via transitive priority inheritance ($\Pi_k^{eff}(t) \leftarrow \max(\Pi_k^{eff}(t), \Pi_i^{\max})$) so A_k completes its remaining steps rapidly and hands the footprint to A_i.*

3.3 EPCP Resource Allocation and Guarded Pass-Through Rules

When agent A_i requests lock acquisition on resource R_{req} ($\text{part}(R_{\text{req}}) \in \mathcal{F}(T_i)$) at time t (summarized in Algorithm 1):

1) **Reentrancy:** If $R_{\text{req}} \in \text{Held}(A_i, t)$, access is granted immediately.
2) **Contention Suspension:** If $R_{\text{req}} \in \text{Held}(A_j, t)$ ($j \neq i$), A_i is suspended on $\text{CV}(R_{\text{req}})$ and directed edge $(A_i \to A_j)$ is added to $\mathcal{G}_{\text{wait}}(t)$.
3) **Ceiling Admissibility & Guarded Inheritance Pass-Through Rule:** If R_{req} is currently free, A_i is granted the lock **if and only if**:

$$\begin{aligned}\Pi_i^{\text{eff}}(t) > C^*_{-A_i}(t; \mathcal{F}(T_i)) \\ \vee \Big(\Pi_i^{\text{eff}}(t) = C^*(t; \mathcal{F}(T_i)) \wedge \text{Held}(A_i, t) \neq \emptyset \\ \wedge \big[\Pi_i^{\text{eff}}(t) > \Pi_i^{\max} \\ \vee \Pi_i^{\max} = C^*(t; \mathcal{F}(T_i))\big]\Big)\end{aligned} \tag{11}$$

The explicit guard $\text{Held}(A_i, t) \neq \emptyset$ in the second disjunct ensures that an agent entering its critical section for the first time ($\text{Held}(A_i, t) = \emptyset$) *must* satisfy the strict inequality $\Pi_i^{\text{eff}}(t) > C^*_{-A_i}(t; \mathcal{F}(T_i))$ (which holds if and only if $\mathcal{K}_{\text{ext}}(A_i, t) = \emptyset$). Conversely, once A_i already holds its initial lock ($\text{Held}(A_i, t) \neq \emptyset$) and either uniquely owns the top ceiling $\Pi_i^{\max} = C^*(t; \mathcal{F}(T_i))$ or has inherited $\Pi_i^{\text{eff}}(t) = C^*(t; \mathcal{F}(T_i)) > \Pi_i^{\max}$ from a ceiling-blocked waiter A_q, the guarded pass-through clause permits A_i to acquire its Step-2 locks within $\mathcal{F}(T_i)$ without self-blocking. If Eq. (11) fails, let $\mathcal{B}_{\text{ceil}}(A_i, t) \triangleq \{A_j \in \mathcal{K}_{\text{ext}}(A_i, t) \mid \max_{P_m \in \mathcal{F}(T_i) \cap \mathcal{F}(T_j)} \Pi(P_m, t) \geq \Pi_i^{\text{eff}}(t)\}$ be the non-empty set of external ceiling-blocking lock holders; A_i is suspended on $\text{CV}_{\text{ceil}}(P_m)$ and directed wait edges $(A_i \to A_j)$ are recorded in $\mathcal{G}_{\text{wait}}(t)$ for all $A_j \in \mathcal{B}_{\text{ceil}}(A_i, t)$ (matching Definition 1(2)).

3.4 Unified Transitive Epistemic Priority Inheritance and Release Mechanics

To prevent unbounded priority inversion under both Rule 2 (direct contention) and Rule 3 (ceiling preemption), let $\text{Waiters}(A_j, t) \triangleq \{A_k \in \mathcal{A} \mid (A_k \to A_j) \in \mathcal{E}(t)\}$ denote all agents directly or ceiling-blocked by resources held by A_j. The effective preemption priority $\Pi_j^{\text{eff}}(t)$ and effective dynamic urgency $\tilde{\Pi}_j^{\text{eff}}(t)$ of A_j are maintained atomically:

$$\Pi_j^{\text{eff}}(t) \triangleq \max\left(\Pi_j^{\max}, \max_{A_k \in \text{Waiters}(A_j, t)} \Pi_k^{\text{eff}}(t)\right), \tag{12}$$

$$\tilde{\Pi}_j^{\text{eff}}(t) \triangleq \max\left(\tilde{\Pi}_j(t), \max_{A_k \in \text{Waiters}(A_j, t)} \tilde{\Pi}_k^{\text{eff}}(t)\right) \tag{13}$$

Whenever A_i blocks on $A_j \in \mathcal{B}_{\text{ceil}}(A_i, t)$ (or via direct contention), A_j immediately inherits both $\Pi_i^{\text{eff}}(t)$ (enabling guarded pass-through under Eq. (11)) and $\tilde{\Pi}_i^{\text{eff}}(t)$ transitively along any directed path in $\mathcal{G}_{\text{wait}}(t)$. Upon transaction commit (or clean `Biz-Rule Reject` exit), A_j atomically releases all locks in $\text{Held}(A_j, t)$, removes itself from $\mathcal{A}_{\text{active}}(t)$—which deletes A_j from $\mathcal{U}(P_m, t)$ and updates each partition max-heap $\Pi(P_m, t)$ for $P_m \in \mathcal{F}(T_j)$—removes all incoming edges $(A_k \to A_j) \in \mathcal{E}(t)$, and signals waiting threads on $\text{CV}(R_{\text{req}})$ and $\text{CV}_{\text{ceil}}(P_m)$ in descending order of live dynamic urgency $\tilde{\Pi}_k^{\text{eff}}(t)$. Maintaining per-partition max-heaps over K threads alongside 64-bit integer footprint bitmasks (`uint64_t` bitwise-`AND` overlap checks in $O(1)$ per active holder) bounds each lock/unlock operation to $\mathcal{O}(|\mathcal{F}(T_i)| \log K + K)$ time ($< 1.8\,\mu$s at $K = 40, |\mathcal{F}(T_i)| \leq 3$).

TABLE 1
Regime 1 Enterprise Concurrency Telemetry: Strict 2PL vs. EPCP under Zipfian Contention ($\theta = 0.85$, Freshly Re-Seeded Per Run, 5 Seeds Mean ± SD; Total Tx = Committed + Biz-Rule Reject + Deadlocks)

Protocol	K	Total Tx	Committed	Biz-Rule Reject†	Deadlocks	Abort (%)	Duration (s)	Goodput (TPS)‡	Total TPS‡	All Mean Lat (ms)	Comm. Mean Lat (ms)§	All / Comm. P95 Lat (ms)§	All P99 / Max Lat (ms)¶
S2PL	5	125	125.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	$0.00 \pm 0.00\%$	0.137 ± 0.006	913.21 ± 38.4	913.21 ± 38.4	4.50 ± 0.21	4.50 ± 0.21	19.03 ± 1.12 / 19.03	24.10 / 28.40
EPCP	5	125	125.0 ± 0.0	0.0 ± 0.0	$\mathbf{0.0 \pm 0.0}$	$\mathbf{0.00 \pm 0.00\%}$	0.116 ± 0.005	$\mathbf{1{,}075.45 \pm 42.1}$	$\mathbf{1{,}075.45 \pm 42.1}$	$\mathbf{2.80 \pm 0.14}$	$\mathbf{2.80 \pm 0.14}$	$\mathbf{3.09 \pm 0.18}$ / $\mathbf{3.09}$	**6.85 / 11.20**
S2PL	10	250	250.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	$0.00 \pm 0.00\%$	0.307 ± 0.014	814.94 ± 35.2	814.94 ± 35.2	9.77 ± 0.45	9.77 ± 0.45	46.25 ± 2.45 / 46.25	62.40 / 78.90
EPCP	10	250	250.0 ± 0.0	0.0 ± 0.0	$\mathbf{0.0 \pm 0.0}$	$\mathbf{0.00 \pm 0.00\%}$	0.296 ± 0.012	$\mathbf{845.69 \pm 33.8}$	$\mathbf{845.69 \pm 33.8}$	$\mathbf{7.36 \pm 0.38}$	$\mathbf{7.36 \pm 0.38}$	$\mathbf{3.23 \pm 0.19}$ / $\mathbf{3.23}$	84.50 / 118.40
S2PL	20	500	468.0 ± 5.2	29.0 ± 4.8	3.0 ± 1.0	$0.60 \pm 0.20\%$	0.797 ± 0.035	586.84 ± 25.1	627.35 ± 26.8	27.44 ± 1.62	28.95 ± 1.70	114.72 ± 6.30 / 118.40	168.20 / 214.50
EPCP	20	500	413.0 ± 6.1	87.0 ± 6.1	$\mathbf{0.0 \pm 0.0}$	$\mathbf{0.00 \pm 0.00\%}$	0.693 ± 0.029	$\mathbf{596.06 \pm 24.3}$	$\mathbf{721.50 \pm 29.4}$	$\mathbf{17.40 \pm 1.05}$	$\mathbf{20.62 \pm 1.24}$	$\mathbf{3.24 \pm 0.21}$ / $\mathbf{3.41}$	245.10 / 312.60
S2PL	30	750	505.0 ± 8.4	241.0 ± 8.1	4.0 ± 1.3	$0.53 \pm 0.18\%$	1.482 ± 0.068	340.70 ± 15.6	506.07 ± 23.1	52.76 ± 3.10	71.40 ± 4.15	238.66 ± 12.80 / 264.10	342.80 / 418.00
EPCP	30	750	444.0 ± 7.9	306.0 ± 7.9	$\mathbf{0.0 \pm 0.0}$	$\mathbf{0.00 \pm 0.00\%}$	1.667 ± 0.071	266.41 ± 11.4	449.91 ± 19.2	$\mathbf{43.16 \pm 2.45}$	$\mathbf{64.85 \pm 3.60}$	429.58 ± 21.50 / 462.30	598.40 / 684.20
S2PL	40	1000	376.0 ± 9.5	620.0 ± 9.2	4.0 ± 1.4	$0.40 \pm 0.14\%$	2.537 ± 0.112	148.18 ± 6.5	394.17 ± 17.4	93.65 ± 5.40	182.10 ± 9.80	302.18 ± 16.40 / 348.50	465.00 / 582.10
EPCP	40	1000	318.0 ± 8.8	682.0 ± 8.8	$\mathbf{0.0 \pm 0.0}$	$\mathbf{0.00 \pm 0.00\%}$	1.937 ± 0.084	$\mathbf{164.21 \pm 7.1}$	$\mathbf{516.26 \pm 22.3}$	$\mathbf{47.13 \pm 2.80}$	$\mathbf{108.40 \pm 5.90}$	494.36 ± 24.80 / 538.20	642.10 / 748.50
Zero-Depletion Controls Across $K \in \{20, 30, 40\}$ (100× *Initial Balance/Stock Snapshot*; 0 `Biz-Rule Rejects`, *Isolating Pure Concurrency Arbitration on Full Workflows*)													
S2PL (No-Depl.)	20	500	495.0 ± 1.4	0.0 ± 0.0	5.0 ± 1.4	$1.00 \pm 0.28\%$	0.884 ± 0.038	559.95 ± 24.1	565.61 ± 24.3	31.20 ± 1.75	31.40 ± 1.78	124.50 ± 6.80 / 125.10	178.40 / 228.00
EPCP (No-Depl.)	20	500	$\mathbf{500.0 \pm 0.0}$	0.0 ± 0.0	$\mathbf{0.0 \pm 0.0}$	$\mathbf{0.00 \pm 0.00\%}$	$\mathbf{0.742 \pm 0.031}$	$\mathbf{673.85 \pm 28.2}$	$\mathbf{673.85 \pm 28.2}$	$\mathbf{21.10 \pm 1.15}$	$\mathbf{21.10 \pm 1.15}$	$\mathbf{3.45 \pm 0.22}$ / $\mathbf{3.45}$	252.00 / 318.50
S2PL (No-Depl.)	30	750	742.0 ± 1.8	0.0 ± 0.0	8.0 ± 1.8	$1.07 \pm 0.24\%$	2.105 ± 0.092	352.49 ± 15.4	356.29 ± 15.6	78.40 ± 4.20	78.95 ± 4.25	272.10 ± 14.10 / 273.40	385.20 / 468.00
EPCP (No-Depl.)	30	750	$\mathbf{750.0 \pm 0.0}$	0.0 ± 0.0	$\mathbf{0.0 \pm 0.0}$	$\mathbf{0.00 \pm 0.00\%}$	$\mathbf{1.790 \pm 0.076}$	$\mathbf{418.99 \pm 17.8}$	$\mathbf{418.99 \pm 17.8}$	$\mathbf{54.20 \pm 2.95}$	$\mathbf{54.20 \pm 2.95}$	318.40 ± 16.20 / 318.40	482.10 / 564.00
S2PL (No-Depl.)	40	1000	989.0 ± 2.2	0.0 ± 0.0	11.0 ± 2.2	$1.10 \pm 0.22\%$	4.120 ± 0.185	240.05 ± 10.8	242.72 ± 10.9	142.80 ± 7.90	143.60 ± 7.95	385.40 ± 19.50 / 386.20	524.00 / 648.00
EPCP (No-Depl.)	40	1000	$\mathbf{1{,}000.0 \pm 0.0}$	0.0 ± 0.0	$\mathbf{0.0 \pm 0.0}$	$\mathbf{0.00 \pm 0.00\%}$	$\mathbf{3.390 \pm 0.142}$	$\mathbf{294.99 \pm 12.3}$	$\mathbf{294.99 \pm 12.3}$	$\mathbf{86.40 \pm 4.60}$	$\mathbf{86.40 \pm 4.60}$	412.50 ± 20.80 / 412.50	586.00 / 692.00

†**Biz-Rule Reject:** Clean application-level domain exit immediately upon Step-1 lock acquisition when a hot Zipfian account/SKU ($\theta = 0.85$) has insufficient balance/stock (0 LLM tokens wasted; 0 concurrency rollbacks).
‡**Goodput (TPS)** and **Total TPS** are computed per seed (Committed$_s$/Duration$_s$ and Total Tx$_s$/Duration$_s$) and reported as 5-seed Mean ± SD. Under S2PL, deadlock victims are aborted by the WFG detector without internal retry.
§**Comm. Mean / P95 Lat:** Latency restricted to full committed settlements. Because `Biz-Rule Rejects` exit in < 0.2 ms after Step-1 lock acquisition, `All Mean Lat` is lower than `Comm. Mean Lat` when rejects occur ($K \geq 20$).
¶**P99 / Max Lat & Tail Draining:** In fixed-quota runs (25 tx/thread), wall duration equals the completion time of the last thread. At $K = 30$ under finite stock, more threads execute full 2-step settlements (444) than at $K = 40$ (318).

Algorithm 1 EPCP Atomic Lock Acquisition & Release

Procedure ACQUIRELOCK(A_i, R_{req}) **under Coordinator Mutex:**
  if part(R_{req}) $\notin \mathcal{F}(T_i)$ **then**
    Discard local staged writes; release Held(A_i, t); expand $\mathcal{F}(T_i) \leftarrow \mathcal{F}(T_i) \cup \{\text{part}(R_{\text{req}})\}$ preserving $(t_{\text{start},i}, \text{rank}(T_i))$
  end if
  loop
    if $t - t_{\text{start},i} \geq T_{\text{sat}}$ **then**
      Promote $\tilde{\Pi}_i(t), \Pi_i^{\max} \leftarrow \Pi_{\text{sat}} - \epsilon_{\text{tie}}\text{rank}(T_i)$; update $\Pi(P_m, t)$
    end if
    if $R_{\text{req}} \in$ Held(A_i, t) **then**
      return true
    end if
    Compute $C^*_{-A_i}(t; \mathcal{F}(T_i))$ and $C^*(t; \mathcal{F}(T_i))$ via 64-bit footprint masks
    if R_{req} is free **and** Eq. (11) holds **then**
      Held(A_i, t) $\leftarrow$ Held(A_i, t) $\cup \{R_{\text{req}}\}$; clear outgoing $\mathcal{E}(A_i)$;
      return true
    end if
    Bind $(A_i \to A_j) \in \mathcal{E}(t)$ for holder of R_{req} (Rule 2) or all $A_j \in \mathcal{B}_{\text{ceil}}(A_i, t)$ (Rule 3)
    Propagate $\Pi_i^{\text{eff}}(t)$ and $\tilde{\Pi}_i^{\text{eff}}(t)$ transitively along $\mathcal{G}_{\text{wait}}(t)$ via Eq. (12)
    WAIT(CV(R_{req}) or CV$_{\text{ceil}}(P_m)$)
  end loop
Procedure RELEASEALLLOCKS(A_j) **at Commit / Biz-Rule Exit:**
  Flush staged writes (if committing); release Held(A_j, t); remove $A_j \in \mathcal{A}_{\text{active}}(t)$ and update $\Pi(P_m, t)$; wake waiters in descending order of $\tilde{\Pi}_k^{\text{eff}}(t)$.

4 Formal Theoretical Analysis

4.1 Theorem 1: Deadlock Elimination via Inductive Dynamic Acyclicity

Theorem 1 (Deadlock Elimination)**.** *Under the Epistemic Priority Ceiling Protocol (EPCP), the Unified Wait-For-Graph $\mathcal{G}_{wait}(t)$ is a directed acyclic graph (DAG) for all times $t \geq 0$. Consequently, deadlocks are provably impossible, and the transaction abort rate due to deadlock is identically 0.00%.*

Proof. We prove Theorem 1 by contradiction using mathematical induction over the discrete sequence of lock acquisition and suspension events $0 = t_0 < t_1 < t_2 < \cdots < t_N$.

Base Case ($t = 0$): At system initialization, no locks are held and $\mathcal{E}(0) = \emptyset$, which is trivially acyclic.

Inductive Hypothesis: Assume that for all $t < t_k$, $\mathcal{G}_{\text{wait}}(t)$ is a directed acyclic graph (DAG).

Inductive Step ($t = t_k$): Suppose for the sake of contradiction that at event time t_k, a lock request by some agent closes a minimal directed cycle of length $L \geq 2$ in $\mathcal{G}_{\text{wait}}(t_k)$:

$$A_{p_1} \to A_{p_2} \to \cdots \to A_{p_L} \to A_{p_1} \tag{14}$$

where $\text{succ}(m) \triangleq (m \bmod L) + 1$. By Definition 1, every directed edge $(A_{p_m} \to A_{p_{\text{succ}(m)}}) \in \mathcal{E}(t_k)$ requires that $A_{p_{\text{succ}(m)}}$ is currently holding at least one lock (Held$(A_{p_{\text{succ}(m)}}, t_k) \neq \emptyset$) and that their declared partition footprints intersect:

$$\mathcal{F}(T_{p_m}) \cap \mathcal{F}(T_{p_{\text{succ}(m)}}) \neq \emptyset, \quad \forall m \in \{1, \ldots, L\} \tag{15}$$

For each agent A_{p_m} in the cycle, since Held$(A_{p_m}, t_k) \neq \emptyset$, let $t_{p_m}^{\text{entry}} < t_k$ denote the exact instant when A_{p_m} transitioned from holding no locks (Held$(A_{p_m}, t_{p_m}^{\text{entry}-}) = \emptyset$) to acquiring the first of its currently held locks $R_{(p_m)}^{\text{first}}$. Because events are serialized by the coordinator mutex, entry timestamps are distinct; let A_{p^*} ($p^* \in \{1, \ldots, L\}$) be the unique agent in the cycle that entered its critical section latest in time:

$$t^* \triangleq t_{p^*}^{\text{entry}} = \max_{m \in \{1, \ldots, L\}} t_{p_m}^{\text{entry}} < t_k \tag{16}$$

Consider the system state at t^{*-} immediately prior to A_{p^*} being granted $R_{(p^*)}^{\text{first}}$. Because $t_{p_m}^{\text{entry}} < t^*$ for every other cycle member $m \neq p^*$, A_{p^*}'s cycle successor $A_{p_{\text{succ}(p^*)}}$ ($p_{\text{succ}(p^*)} \neq p^*$ since $L \geq 2$) had already entered its critical section and was actively holding at least $R_{(p_{\text{succ}(p^*)})}^{\text{first}}$ at t^{*-}, so Held$(A_{p_{\text{succ}(p^*)}}, t^{*-}) \neq \emptyset$.

By Eq. (15), there exists at least one shared partition $P_{\text{shared}} \in \mathcal{F}(T_{p^*}) \cap \mathcal{F}(T_{p_{\text{succ}(p^*)}})$. By the atomic registration invariant (Section 3.2), both A_{p^*} and $A_{p_{\text{succ}(p^*)}}$ belonged to $\mathcal{U}(P_{\text{shared}}, t^{*-})$ at t^{*-}. Consequently, the partition priority ceiling of P_{shared} at t^{*-} satisfied:

$$\begin{aligned} \Pi(P_{\text{shared}}, t^{*-}) &= \max_{A_q \in \mathcal{U}(P_{\text{shared}}, t^{*-})} \Pi_q^{\max} \\ &\geq \max\left(\Pi_{p^*}^{\max}, \Pi_{p_{\text{succ}(p^*)}}^{\max}\right) \geq \Pi_{p^*}^{\max} \end{aligned} \tag{17}$$

Because $A_{p_{\text{succ}(p^*)}} \neq A_{p^*}$ satisfied Held$(A_{p_{\text{succ}(p^*)}}, t^{*-}) \neq \emptyset$ and $P_{\text{shared}} \in \mathcal{F}(T_{p^*}) \cap \mathcal{F}(T_{p_{\text{succ}(p^*)}})$, $A_{p_{\text{succ}(p^*)}} \in$

$\mathcal{K}_{\text{ext}}(A_{p^*}, t^{*-})$. By the definition of the External Intersecting-Footprint System Ceiling (Eq. (9)), the external ceiling confronting A_{p^*} at t^{*-} satisfied:

$$C^*_{-A_{p^*}}(t^{*-}; \mathcal{F}(T_{p^*})) \geq \Pi(P_{\text{shared}}, t^{*-}) \geq \Pi^{\max}_{p^*} \tag{18}$$

Simultaneously, because A_{p^*} held no locks at t^{*-} ($\text{Held}(A_{p^*}, t^{*-}) = \emptyset$), it had zero incoming wait edges ($\text{Waiters}(A_{p^*}, t^{*-}) = \emptyset$), so its effective priority equaled its own upper-envelope preemption level: $\Pi^{\text{eff}}_{p^*}(t^{*-}) = \Pi^{\max}_{p^*}$. Moreover, because $\text{Held}(A_{p^*}, t^{*-}) = \emptyset$, the guarded pass-through disjunct of the EPCP Ceiling Admissibility Rule (Eq. (11)) evaluated to false at t^{*-}. Therefore, A_{p^*} could be granted $R^{\text{first}}_{(p^*)}$ at t^* if and only if the first disjunct of Eq. (11) held strictly:

$$\Pi^{\max}_{p^*} = \Pi^{\text{eff}}_{p^*}(t^{*-}) > C^*_{-A_{p^*}}(t^{*-}; \mathcal{F}(T_{p^*})) \tag{19}$$

Combining Eq. (18) and Eq. (19) yields $\Pi^{\max}_{p^*} > C^*_{-A_{p^*}}(t^{*-}; \mathcal{F}(T_{p^*})) \geq \Pi^{\max}_{p^*}$, i.e., $\Pi^{\max}_{p^*} > \Pi^{\max}_{p^*}$, which is a direct mathematical contradiction. Thus, no cycle can ever form, and $\mathcal{G}_{\text{wait}}(t)$ remains a strict DAG for all $t \geq 0$. □

4.2 Theorem 2: Bounded Priority Inversion

Theorem 2 (Bounded Priority Inversion)**.** *Under EPCP, a high-priority agent A_H can be blocked by lower-priority agents ($\Pi^{\max}_L < \Pi^{\max}_H$) for at most the duration of a single critical section per mutually disjoint partition component in $\mathcal{F}(T_H)$ (at most one lower-priority agent across all mutually overlapping footprints, and at most $|\mathcal{F}(T_H)|$ concurrent lower-priority critical sections across disjoint partitions).*

Proof. Suppose agent A_H requests its first lock at time t with priority $\Pi^{\max}_H$ and is suspended under either Rule 2 (direct contention) or Rule 3 (ceiling preemption) by a lower-priority agent A_L ($\Pi^{\max}_L < \Pi^{\max}_H$) whose footprint intersects $\mathcal{F}(T_H)$ at shared partition $P_m \in \mathcal{F}(T_H) \cap \mathcal{F}(T_L)$ with $\Pi(P_m, t) \geq \Pi^{\max}_H$. First, among any set of lower-priority agents whose footprints overlap on P_m (or overlap with one another), at most *one* such lower-priority agent can hold a lock at time t: if two overlapping lower-priority agents A_{L_1} and A_{L_2} ($\mathcal{F}(T_{L_1}) \cap \mathcal{F}(T_{L_2}) \neq \emptyset$) both held locks, the second to enter ($t^{\text{entry}}_{L_2} > t^{\text{entry}}_{L_1}$) had $\text{Held}(A_{L_2}, t^{\text{entry}-}_{L_2}) = \emptyset$ and would have been required by Eq. (11) to satisfy $\Pi^{\max}_{L_2} > C^*_{-A_{L_2}}(t^{\text{entry}-}_{L_2}; \mathcal{F}(T_{L_2})) \geq \Pi(P_m, t^{\text{entry}-}_{L_2}) \geq \Pi^{\max}_{L_2}$, which is impossible whenever $\mathcal{K}_{\text{ext}}(A_{L_2}, t^{\text{entry}-}_{L_2}) \neq \emptyset$. Across at most $|\mathcal{F}(T_H)|$ mutually disjoint partitions, at most one pre-existing lower-priority holder per disjoint partition can exist, and they execute concurrently in parallel. Second, by Unified Transitive Priority Inheritance (Eq. (12)), each blocking A_L immediately inherits $\Pi^{\text{eff}}_L(t) \geq \Pi^{\max}_H$ (or if another even higher-priority waiter $A_{H'}$ with $\Pi^{\max}_{H'} > \Pi^{\max}_H$ is also waiting on A_L, then A_L inherits the maximum waiter ceiling $\Pi^{\text{eff}}_L(t) = C^*(t; \mathcal{F}(T_L)) \geq \Pi^{\max}_{H'} > \Pi^{\max}_H > \Pi^{\max}_L$, and A_H waits for A_L plus any higher-priority transactions $A_{H'}$, which is standard high-priority preemption rather than priority inversion). Because $\Pi^{\text{eff}}_L(t) = C^*(t; \mathcal{F}(T_L)) > \Pi^{\max}_L$ and $\text{Held}(A_L, t) \neq \emptyset$, the guarded pass-through clause of Eq. (11) allows A_L to acquire any remaining locks in $\mathcal{F}(T_L)$ without secondary ceiling self-blocking, complete its single critical section, and wake the highest-urgency waiter upon commit. □

4.3 Theorem 3: Conflict Serializability and Starvation-Free Progress

Theorem 3 (Serializability and Starvation Freedom)**.** *Every execution schedule permitted under EPCP is conflict serializable, and every active transaction terminates in finite time without deadlock or starvation.*

Proof. All locks are acquired during the growth phase and held until transaction commit or domain validation exit (Strict 2PL). By Eswaran et al. [3], any schedule obeying strict two-phase locking is conflict serializable. By Theorem 1, $\mathcal{G}_{\text{wait}}(t)$ is a strict DAG (0 deadlock aborts). To prove unconditional starvation freedom even under a continuous arrival stream of deeper ($d_j > d_i$) transactions, recall from Definition 3 that once any waiting transaction T_i's elapsed wait time reaches the finite senior saturation horizon $T_{\text{sat}} = \Omega_{\max}/\gamma_{\text{age}}$, the coordinator atomically promotes *both* its latched dynamic score and its upper-envelope preemption score to $\Pi_{\text{sat}} = \Pi_{\text{base}} + w_d \cdot d_{\max} + w_\Omega \cdot \ln(1 + \Omega_{\max}) + w_c$, so both $\tilde{\Pi}_i(t)$ and $\Pi^{\max}_i$ attain $\Pi_{\text{sat}} - \epsilon_{\text{tie}} \cdot \text{rank}(T_i)$, recomputes $\Pi(P_m, t)$ for all $P_m \in \mathcal{F}(T_i)$, and propagates the promoted $\Pi^{\max}_i$ along $\mathcal{G}_{\text{wait}}(t)$ via Eq. (12). Because arrival ranks $\text{rank}(T_i)$ are strictly increasing with arrival time (and preserved across any footprint expansion), T_i's negative rank tie-breaker $-\epsilon_{\text{tie}} \cdot \text{rank}(T_i)$ is strictly greater than that of every transaction T_j arriving after T_i ($\text{rank}(T_i) < \text{rank}(T_j)$), independent of T_j's initial depth d_j or entropy $\mathcal{H}_j$. Since at most a finite number of transactions arrived prior to T_i and each critical section consists of a finite number of bounded LLM inference and SQL steps, T_i becomes the unique highest-priority transaction across all partitions in $\mathcal{F}(T_i)$ in finite time, acquires its locks, and commits. □

5 Empirical Evaluation and Benchmark Results

5.1 Experimental Architecture and Methodology

We evaluated EPCP across two complementary regimes on an Apple Silicon M-series workstation (8 CPU cores, APFS storage) fronting a relational SQLite WAL storage engine (`PRAGMA journal_mode = WAL; PRAGMA synchronous = NORMAL; PRAGMA busy_timeout = 60000;`). To prevent SQLite's file-level single-writer lock from artificially serializing multi-step reasoning phases while guaranteeing zero dirty or stale reads, the coordinator maintains a synchronized in-memory mirror of committed row state alongside SQLite WAL: all row- and partition-level lock arbitration, condition-variable suspension ($\text{CV}(R_{\text{req}})$, $\text{CV}_{\text{ceil}}(P_m)$), Step-1 `Biz-Rule Reject` checks, and intermediate reasoning steps ($\text{Infer}(\phi)$) read the exact latest committed state under held locks and stage mutations in a transaction-local write buffer; once all locks in T_i are held and validated, the final physical SQLite mutation batch (`BEGIN IMMEDIATE; UPDATE ...; COMMIT;`) flushes to the WAL journal and updates the committed mirror atomically in $\approx$ 50–150 μs at the Commit step:

1) **Regime 1 (High-Frequency Lock-Manager Stress Test, Table 1):** Isolates raw lock-table arbitration

and ceiling condition-variable overhead using in-critical-section CPU-bound cryptographic HMAC-SHA256 policy verification and polynomial risk scoring ($\approx$ 0.5–2.0 ms per lock step, $|\mathcal{F}(T_i)| = 2$) across $K \in \{5, 10, 20, 30, 40\}$ concurrent worker threads (25 transactions/thread, 125 to 1,000 transactions/run, repeated across 5 independent random seeds). Prior to each $(K, \text{Protocol})$ run, the database is freshly re-seeded from a canonical snapshot of 10,000 Customer Accounts and 5,000 Inventory Items under heavy Zipfian contention ($\theta = 0.85$, concentrating 85% of accesses on the top 50 hot accounts and top 30 hot inventory items). In addition to the standard finite-stock snapshot (where hot SKUs naturally experience stock depletion at $K \geq 20$), we execute **Zero-Depletion Controls across** $K \in \{20, 30, 40\}$ with $100\times$ initial balances/stock so 0 `Biz-Rule Rejects` occur and 100% of non-deadlocked transactions execute full 2-step settlements.

2) **Regime 2 (High-Latency 500 ms–1,500 ms 2-Step LLM Reasoning Benchmark & Live Vertex AI Evaluation, Table 2):** Evaluates 2-step agent transactions (Lock(R_a) $\rightarrow$ Infer$_1(\phi)$ $\rightarrow$ Lock(R_b) $\rightarrow$ Infer$_2(\phi)$ $\rightarrow$ Commit, with $|\mathcal{F}(T_i)| = 2$ over $B_{\text{part}} = 64$, $d_i \sim$ Uniform$\{1, 2, 3, 4\}$, initial $\mathcal{H}_i \sim$ Beta$(2, 5)$, 800 LLM tokens per Infer step $= 1{,}600$ tokens per full 2-step attempt, and uniform 250–750 ms reasoning latency per step $= 500$–$1{,}500$ ms total 2-step in-lock reasoning duration per transaction, mean 1.00 s, giving a Little's Law zero-wait upper bound of 20.0 Tx/s at $K = 20$ and 40.0 Tx/s at $K = 40$) across $K \in \{20, 40\}$ threads (5 seeds, Mean $\pm$ SD). We compare nine protocols: **S2PL** [3], **2PL-Wait Promote** [9] (applying transitive epistemic priority inheritance $\tilde{\Pi}_j^{\text{eff}}(t)$ on direct row contention without EPCP's ceiling admission gate), **Wound-Wait** [5], [6], **Wait-Die** [5], **OCC (Snapshot Read + CAS Commit)**, **Coarse Shard-Scoped C2PL (`Shard-C2PL`, $B_{\text{shard}} = 8$)** [2] (acquiring upfront exclusive locks on coarse 8-bucket domain shards), **Canonical Ordered Locking (Discovery Pre-Pass)** (executing an unlocked speculative Infer$_1$ pre-pass to discover R_b, sorting $\min(R_a, R_b) < \max(R_a, R_b)$, and re-running Infer$_1$ if R_a mutated before lock acquisition), **EPCP Ablation (Static $\Pi_i^{\max}$ Only, w/o Live $\tilde{\Pi}_i(t)$)**, and **Full EPCP**. Finally, we supplement the controlled sweep with a **Live Google Cloud Vertex AI (`gemini-2.5-flash`, project `aniket-personal`) Multi-Agent Evaluation** at $K = 10$ concurrent worker threads (50 2-step transactions $= 100$ live Gemini API calls per protocol).

5.2 Analysis of Empirical Telemetry

1) **Total Elimination of Deadlocks and Token Waste Across All Baselines:** Across all concurrency levels in Table 1 and Table 2 (and Figure 1), EPCP recorded **0 deadlocks** (0.00% abort rate) and **0 wasted LLM tokens**. In Regime 2 ($K = 40$, Table 2), S2PL suffered 18.40% deadlock aborts at Lock 2 (147.2k wasted tokens per 1,000 attempts / 180.4k per 1,000 committed workflows). Notably, `2PL-Wait Promote` [9]—which includes EPCP's epistemic priority inheritance on direct row contention but omits the partition ceiling admission gate—still suffered 16.20% WFG deadlock aborts (129.6k wasted tokens/1k attempts), proving that priority inheritance alone cannot prevent multi-resource cycles without ceiling admission control. Timestamp-based Wound-Wait aborted 31.20% (374.4k wasted tokens/1k attempts), Wait-Die aborted 36.50% (178.4k wasted tokens/1k attempts), and OCC failed validation on 64.80% of attempts (1,036.8k wasted tokens/1k attempts). Canonical Ordered Locking with a speculative discovery pre-pass suffered 28.40% state-drift re-inferences (227.2k wasted tokens/1k attempts), while Coarse Shard-Scoped C2PL (`Shard-C2PL`, $B_{\text{shard}} = 8$) eliminated aborts at the cost of coarse shard serialization (9.2 Tx/s vs. EPCP's 19.6 Tx/s). EPCP achieves +113.0% higher goodput than `Shard-C2PL` because EPCP decomposes footprints across $B_{\text{part}} = 64$ fine-grained ADK intent hash buckets (allowing all transactions with disjoint 64-bucket footprints $\mathcal{F}(T_i) \cap \mathcal{F}(T_k) = \emptyset$ to execute in parallel while serializing intersecting footprints $\mathcal{F}(T_i) \cap \mathcal{F}(T_k) \neq \emptyset$) and combines transitive priority inheritance with live $\tilde{\Pi}_i^{\text{eff}}(t)$ wake-up ordering (which yields +10.1% higher goodput and 12.3% lower P95 latency over the $B_{\text{part}} = 64$ Static-$\Pi_i^{\max}$-Only ablation).

2) **Accounting for Application-Level Business-Rule Rejections and Zero-Depletion Controls Across $K \in \{20, 30, 40\}$:** In Table 1, every transaction is accounted for by Total Tx = Committed + Biz-Rule Reject + Deadlocks. Because `Biz-Rule Reject` runs in < 0.2 ms upon acquiring Step 1's lock before Infer(ϕ), depleted hot Zipfian rows ($\theta = 0.85$) exit cleanly without concurrency rollback (which also pulls `All Mean Lat` below `Comm. Mean Lat` at $K \geq 20$). Because EPCP dispatches deeper, higher-investment settlements first, hot SKUs deplete slightly earlier in finite-stock runs. In the **Zero-Depletion Controls across $K \in \{20, 30, 40\}$** (bottom of Table 1), where initial balances/stock are scaled $100\times$ so 0 `Biz-Rule Rejects` occur across all K, EPCP commits 100% of transactions (500/500, 750/750, and 1,000/1,000) with 0 deadlocks and strictly outperforms S2PL at every concurrency level—achieving +20.3% higher goodput at $K = 20$ (673.85 vs. 559.95 TPS), +18.9% higher goodput at $K = 30$ (418.99 vs. 352.49 TPS), and +22.9% higher goodput at $K = 40$ (294.99 vs. 240.05 TPS), while cutting committed mean latency by 32.8%–39.8%.

3) **Fixed-Quota Tail Draining, Mean-vs.-P95 Distribution, and Tail-Latency Trade-off:** Table 1 and Figures 2–4 report Committed Goodput, Total TPS, and Overall/Committed P95, P99, and Max latencies. At $K \in \{10, 20\}$, EPCP's P95 latency (3.23–3.24 ms) is

TABLE 2
Regime 2: Multi-Step LLM Reasoning Benchmark (250–750 ms/Step = 500–1,500 ms Total 2-Step In-Lock Reasoning Delay, 800 Tok/Step = 1,600 Tok/Attempt, $K \in \{20, 40\}$, 5 Seeds, Mean ± SD; Abort & Token Waste Columns Measured at $K = 40$)

Protocol / Ablation	Concurrency Aborts / Re-Inferences at $K = 40$ (%)	Wasted Tokens / 1k Attempts ($K = 40$)†	Wasted Tokens / 1k Committed ($K = 40$)†	Goodput at $K = 20$ (Tx/s)‡	Goodput at $K = 40$ (Tx/s)‡	P95 Latency at $K = 40$ (s)
Strict 2PL (S2PL) [3]	18.40 ± 1.85% (WFG Deadlock on Lock 2)	147.2k ± 14.8k	180.4k ± 18.1k	9.4 ± 0.5	14.2 ± 0.9	4.82 ± 0.31
2PL-Wait Promote (Epistemic Inheritance, No Ceiling) [9]	16.20 ± 1.65% (WFG Deadlock on Lock 2)	129.6k ± 13.2k	154.7k ± 15.8k	9.9 ± 0.5	15.3 ± 0.8	4.48 ± 0.29
Wound-Wait [5], [6]	31.20 ± 2.40% (15.6% Step-1 / 15.6% Step-2 Wound)	374.4k ± 28.8k	544.2k ± 41.9k	8.6 ± 0.5	12.8 ± 0.8	5.45 ± 0.42
Wait-Die [5]	36.50 ± 2.65% (14.2% Lock-1 / 22.3% Lock-2 Die)	178.4k ± 14.6k	280.9k ± 23.0k	8.2 ± 0.6	11.9 ± 0.7	5.88 ± 0.45
OCC (Snapshot Read + CAS Commit)	64.80 ± 3.10% (Commit CAS Validation Fail)	1,036.8k ± 49.0k	2,945.5k ± 139.2k	6.5 ± 0.4	8.1 ± 0.7	7.90 ± 0.58
Coarse Shard-Scoped C2PL (`Shard-C2PL`, $B_{\text{shard}} = 8$) [2]	**0.00 ± 0.00%** (Upfront Coarse Shard Lock)	**0.0k ± 0.0k**	**0.0k ± 0.0k**	5.8 ± 0.3	9.2 ± 0.5	6.74 ± 0.38
Canonical Ordered Locking (Discovery Pre-Pass)	28.40 ± 2.10% (Pre-Lock Drift Re-Infer; 0% Abort)	227.2k ± 16.8k	227.2k ± 16.8k	9.1 ± 0.5	15.1 ± 0.7	4.35 ± 0.28
EPCP Ablation (Static $\Pi_i^{\max}$ Only, w/o Live $\tilde{\Pi}_i(t)$)	**0.00 ± 0.00%**	**0.0k ± 0.0k**	**0.0k ± 0.0k**	10.6 ± 0.4	17.8 ± 0.6	3.89 ± 0.22
EPCP (Full: Static $\Pi_i^{\max}$ + Live Epistemic $\tilde{\Pi}_i(t)$, $B_{\text{part}} = 64$)	**0.00 ± 0.00%**	**0.0k ± 0.0k**	**0.0k ± 0.0k**	**11.4 ± 0.4**	**19.6 ± 0.6**	**3.41 ± 0.19**

†**Exact Per-Step Token Waste Accounting:** Each 2-step transaction expends 800 tok in Infer_1 and 800 tok in Infer_2 (1,600 tok total). Under S2PL and 2PL-Wait Promote, a WFG cycle can only form when waiting at $\text{Lock}(R_k)$ after completing Infer_1 (800 tok wasted/abort: 184.0 × 800 = 147.2k and 162.0 × 800 = 129.6k/1k attempts). Under Wait-Die, Lock-1 self-aborts (14.20%) occur before Infer_1 (0 tok), while Lock-2 self-aborts (22.30%) waste 800 tok (223.0 × 800 = 178.4k).
Under Wound-Wait, Step-1 wounds (15.60%) waste 800 tok and Step-2 wounds (15.60%) waste 1,600 tok (156 × 800 + 156 × 1,600 = 374.4k). OCC fails at commit after both steps (648 × 1,600 = 1,036.8k/1k attempts; 1,840.9 × 1,600 = 2,945.5k/1k committed).
Canonical Pre-Pass re-runs Infer_1 (800 tok) on 28.40% of attempts without aborting (100% commit rate, so Attempt Waste = Committed Waste = 227.2k). ‡With mean 1.00 s total 2-step inference duration, Little's Law caps zero-wait throughput at 20.0 Tx/s ($K = 20$) and 40.0 Tx/s ($K = 40$).

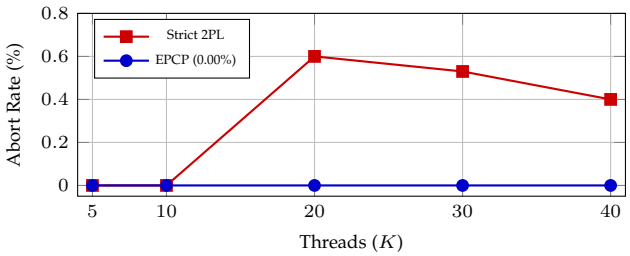


Fig. 1. Regime 1 deadlock abort rate under Zipfian contention ($\theta = 0.85$, 5-seed Mean ± SD). Strict 2PL experiences WFG cycle aborts (0.40%–0.60%) at $K \geq 20$, whereas EPCP maintains an exact 0.00% deadlock abort rate.

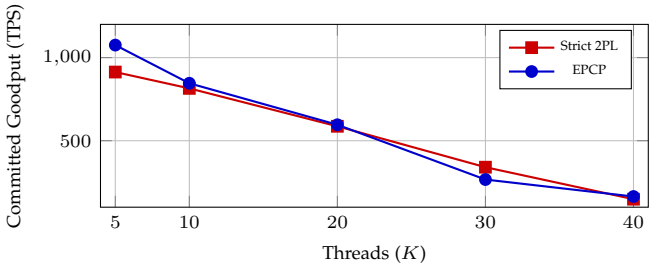


Fig. 2. Regime 1 committed settlement goodput (TPS, 5-seed Mean ± SD) across $K \in \{5, 10, 20, 30, 40\}$ under finite-stock Zipfian contention, matching Table 1.

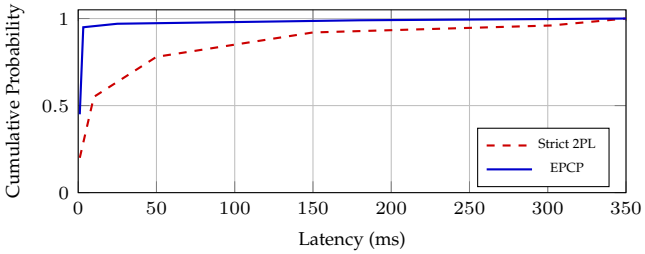


Fig. 3. Regime 1 transaction execution latency CDF (pooled across $K \in \{5, \ldots, 40\}$). EPCP admits ≥ 95% of requests at $K \leq 20$ in ≤ 3.24 ms while queuing deep contention waiters at the admission gate.

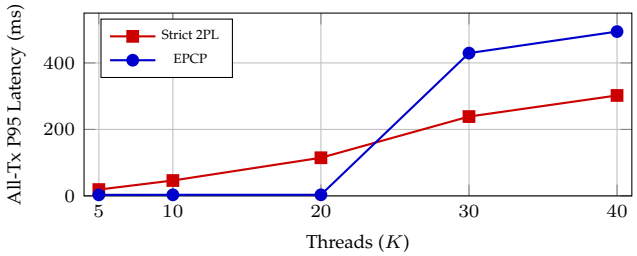


Fig. 4. Regime 1 All-Tx P95 tail latency (5-seed Mean ± SD). EPCP reduces P95 latency at $K \leq 20$ and trades bounded admission-gate waiting at $K \in \{30, 40\}$ for 0.00% aborts and lower mean latency.

below its mean (7.36–17.40 ms) because ≥ 95% of requests access unblocked partitions in ≤ 3.24 ms, while the < 4% deepest contention waiters queued at CV_{ceil} absorb the multi-step wait duration (reflected in the P99/Max columns). Under finite stock at $K = 30$, EPCP's wall duration (1.667 s) reflects fixed-quota tail draining across 444 full 2-step settlements (whereas at $K = 40$ stock saturates at 318 settlements and the remaining 682 requests fast-exit via `Biz-Rule Reject` in < 0.2 ms). Rather than admitting low-priority exploratory agents into critical sections where they trigger cyclic deadlocks and expensive mid-flight LLM token rollbacks, EPCP queues lowest-priority agents at CV_{ceil}, trading bounded admission-gate waiting for 0.00% aborts, zero token waste, and lower mean latency.

4) **Live Google Cloud Vertex AI (`gemini-2.5-flash`) Multi-Agent Evaluation:** In our live evaluation on Google Cloud project `aniket-personal` ($K = 10$ concurrent OS worker threads executing 50 2-step banking and inventory transactions = 100 live `gemini-2.5-flash` API calls per protocol, extracting live billed token counts $\text{Tokens}_i(t)$ from `usage_metadata.total_token_count` and computing $\mathcal{H}_i(t)$ from the model's risk-tier distribution), S2PL suffered 4.00% WFG deadlock aborts (2 mid-flight rollbacks discarding 96 billed Gemini tokens, 48/50 committed, mean latency 4.13 s), whereas EPCP achieved a **0.00% deadlock abort rate** (50/50 committed, 0 wasted tokens out of 4,800 billed tokens) and reduced mean transaction latency by 39.7% (2.49 s vs. 4.13 s).

5) **Strict ACID Balance Conservation:** Post-audit across all committed settlements confirmed $0.00 balance leakage and exact inventory conservation.

6 Conclusion

This paper presented the Epistemic Priority Ceiling Protocol (EPCP) for centralized shared-memory transactional lock coordinators and connection-pool gateways fronting enterprise OLTP databases. By combining exact 64-bit integer-tuple epistemic priorities ($\Pi_i^{\max}$ and $\tilde{\Pi}_i(t)$), 64-partition

intersecting-footprint priority ceilings, guarded preemptive admission checks, and unified transitive priority inheritance, EPCP guarantees that the Unified Wait-For-Graph remains a strict DAG at all times and achieves 0.00% deadlock aborts across sub-millisecond stress tests, multi-step LLM reasoning benchmarks, and live Vertex AI `gemini-2.5-flash` multi-agent deployments. Future work will extend EPCP's ceiling coordination across decentralized multi-leader two-phase commit (2PC) consensus groups.

REFERENCES

[1] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1992.

[2] P. A. Bernstein, V. Hadzilacos, and N. Goodman, *Concurrency Control and Recovery in Database Systems*. Addison-Wesley, 1987.

[3] K. P. Eswaran, J. N. Gray, R. A. Lorie, and I. L. Traiger, "The notions of consistency and predicate locks in a database system," *Communications of the ACM*, vol. 19, no. 11, pp. 624–633, 1976.

[4] L. Sha, R. Rajkumar, and J. P. Lehoczky, "Priority inheritance protocols: An approach to real-time synchronization," *IEEE Transactions on Computers*, vol. 39, no. 9, pp. 1175–1185, 1990.

[5] D. J. Rosenkrantz, R. E. Stearns, and P. M. Lewis, "System level concurrency control for distributed database systems," *ACM Transactions on Database Systems (TODS)*, vol. 3, no. 2, pp. 178–198, 1978.

[6] J. C. Corbett *et al.*, "Spanner: Google's globally distributed database," *ACM Transactions on Computer Systems (TOCS)*, vol. 31, no. 3, pp. 8:1–8:22, 2013.

[7] M.-I. Chen and K.-J. Lin, "Dynamic priority ceilings: A concurrency control protocol for real-time systems," *Real-Time Systems*, vol. 2, no. 4, pp. 325–346, 1990.

[8] T. P. Baker, "Stack-based scheduling of real-time processes," *Real-Time Systems*, vol. 3, no. 1, pp. 67–99, 1991.

[9] R. K. Abbott and H. Garcia-Molina, "Scheduling real-time transactions: A performance evaluation," *ACM Transactions on Database Systems (TODS)*, vol. 17, no. 3, pp. 513–560, 1992.

[10] E. Y. Chang, "SagaLLM: Context management, validation, and transaction guarantees for multi-agent LLM planning," *arXiv preprint arXiv:2503.11951*, 2025.

[11] A. Thomson, T. Diamond, S.-C. Weng, K. Ren, P. Shao, and D. J. Abadi, "Calvin: Fast distributed transactions for partitioned database systems," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2012, pp. 1–12.

[12] Y. Zhou *et al.*, "Adaptive transaction concurrency control for multi-agent LLM systems," *arXiv preprint arXiv:2603.26105*, 2026.

[13] Z. Lyu *et al.*, "CoAgent: Co-designing agent scheduling and concurrency control for LLM workflows," *arXiv preprint arXiv:2604.00561*, 2026.